

temperature=0 means the answer is
always identical, right?



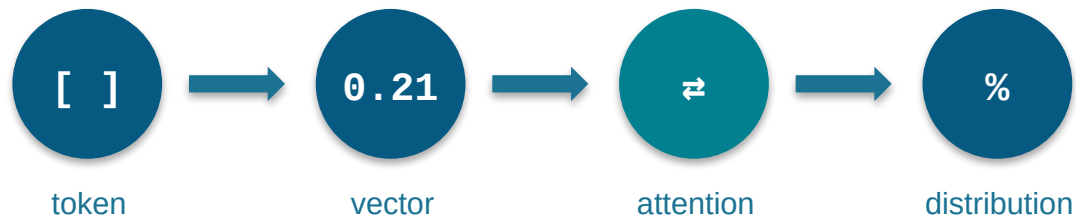
Yes, formally deterministic. But no — not in practice. Why — today's topic.

LECTURE 2

How Modern Large Language Models Work

02

The inference pipeline — and the boundaries that change engineering decisions



Lecture map — 7 sections

0 **Introduction** — *the frame and the pipeline as a whole*

1 **Tokenization** — *how the model sees your text*

2 **Embeddings** — *the space of meaning and the boundary of similarity*

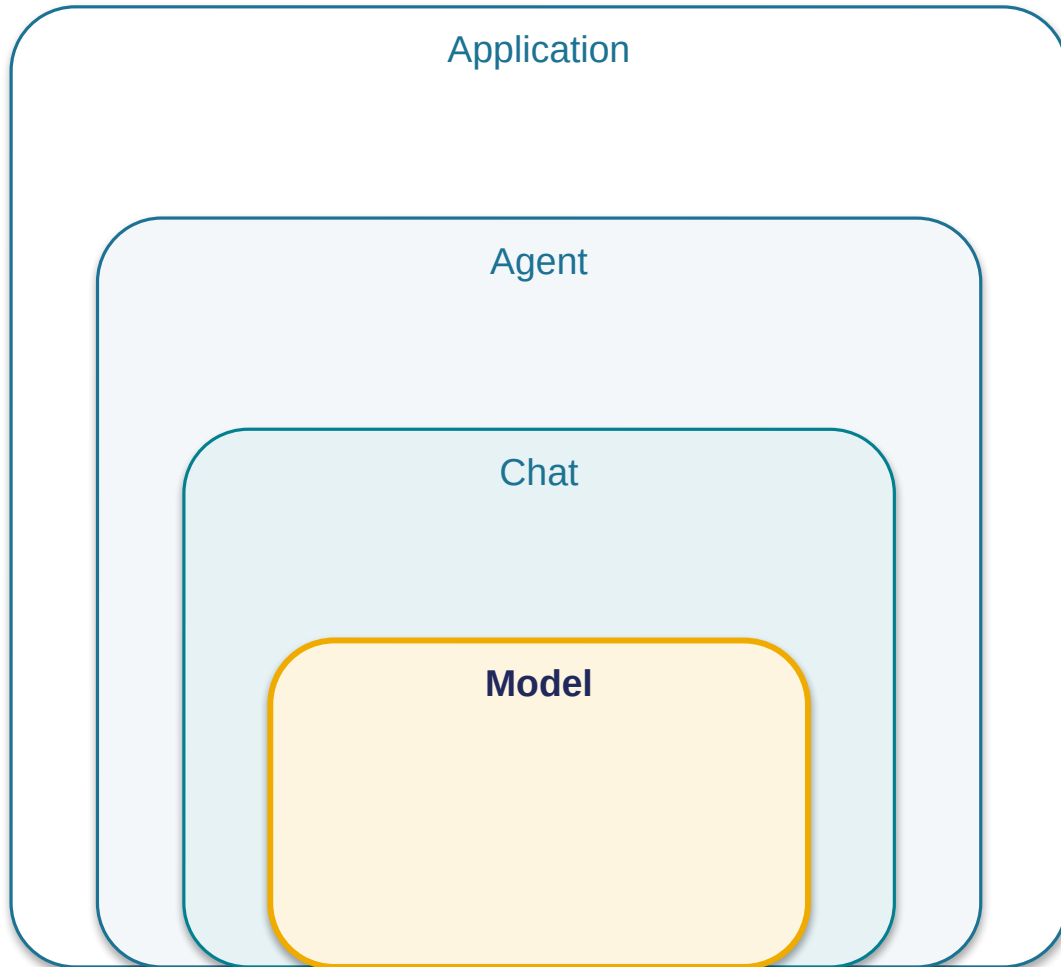
3 **Attention Mechanism** — *what matters right now: roles, caching, long context*

4 **Sampling** — *from distribution to token: temperature, determinism, invisible tokens*

5 **Model Types and Sizes** — *what models run on, multimodality, the 2026 landscape*

6 **Wrap-up** — *assembling the pipeline, recap of the mechanisms*

Today's object — the “model” layer from Lecture 1's four layers



The “model” layer: stateless inference — input goes in, a prediction comes out, no memory between calls.

Today: we take apart what happens inside that inference — and where its design changes engineering decisions.

Lecture goal

“Examine how a language model works — and get into the **important details** that change how you build prompts, agents, and decisions.”

why a fixed “strawberry” answer proves nothing

why a role-based prompt genuinely changes the answer

what a 1M-token window can actually do

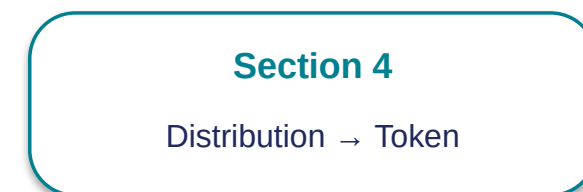
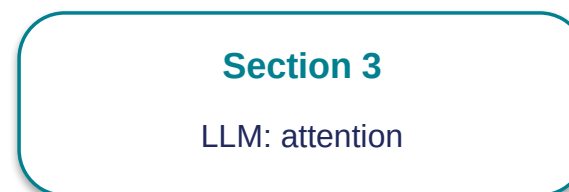
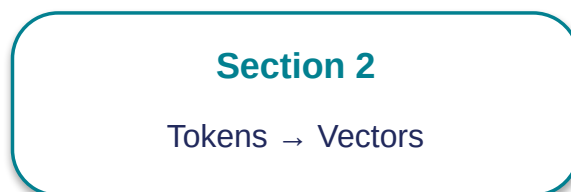
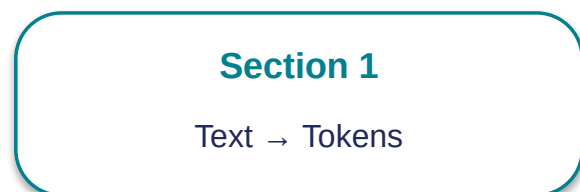
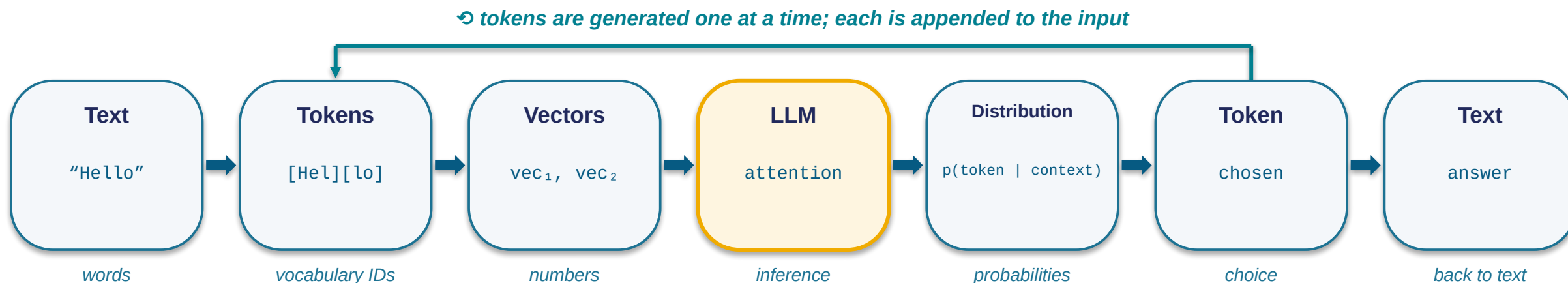
why $T=0$ doesn't guarantee identical answers

what invisible reasoning tokens actually cost

small model vs. giant — what criterion to choose by

what to replace blind trust in benchmarks with

Data flow in an LLM — there and back again



Words only exist at the edges; inside — vectors.

Section 1

Tokenization

“How the model sees your text”

3 case studies · 3 failures



the model confidently gives a wrong answer — it sees chunks, not letters



A token is an ID from the model's vocabulary — not a letter, not a word

cat → [cat] → 1 token

hyperparameter → [hyper][param][eter] → 3 tokens

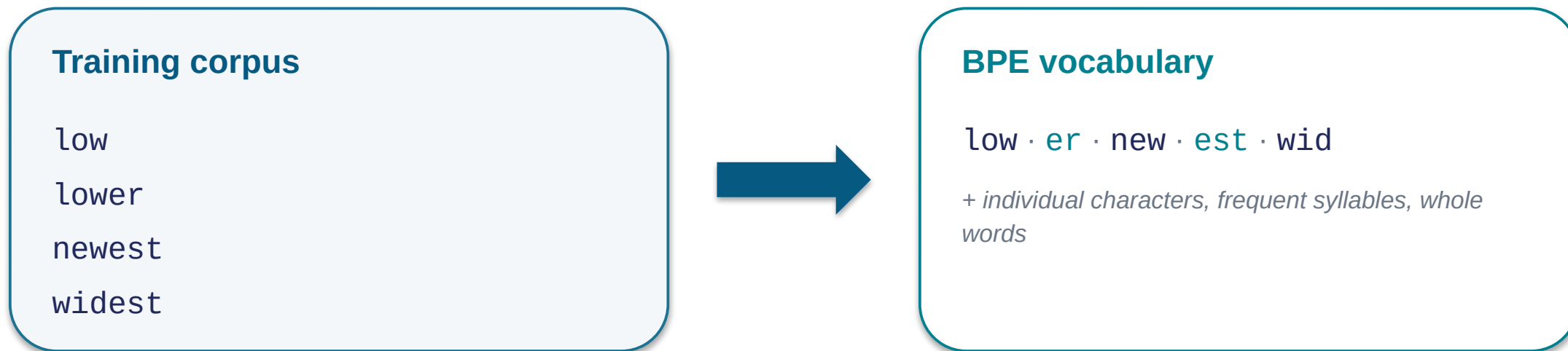
tokenization → [token][ization] → 2 tokens (o200k_base)

“On average: 1 token $\approx$ 4 characters in English ($\approx$ 2 characters in Russian)”

Vocabulary and model are two separate artifacts: the vocabulary is built before the model is trained, by a separate algorithm on its own corpus.

BPE — a compromise between an alphabet and a full-word vocabulary

Not all letters (too long) and not all words (unfamiliar ones fall through) — frequent subsequences.



The vocabulary is built once, before training; at inference — a lookup of ready-made rules, not a computation.

Different vendors cut the same text differently: Claude, GPT, Gemini all have their own vocabularies and rules.

The patch race: they fixed strawberry, then cranberry — one word at a time

Mechanism — letter blindness

strawberry → [st][raw][berry] The model sees **3 tokens**, not 10 letters.

GPT-5.2: 2 r's in strawberry	
GPT-5.5: strawberry ✓ / cranberry ✗	
GPT-5.6: cranberry ✓	
StrawberryBench: still 60%	

GPT-5.2 · Dec 2025

strawberry ✗ — “two r's in strawberry”

GPT-5.5 · Apr 2026

strawberry ✓ / cranberry ✗ — “two r's” instead of three

GPT-5.6 · Jul 2026

cranberry ✓ — another viral case patched

StrawberryBench

847 questions, 7 difficulty levels — systematic testing instead of one viral question

“Jagged intelligence”: skill level is set by training data, not by “general intelligence” — the same model wins Olympiad gold and fails letter counting.

What to do: test your own domain's “cranberry” — obscure cases nobody hyped about; going viral ≠ having the skill.

The tokenizer cuts by frequency, not by structure

Numbers and code are the most common “non-text” inputs: the model's arithmetic and your token budget both depend on how they're cut.

Numbers

1000000 → [100][000][0]

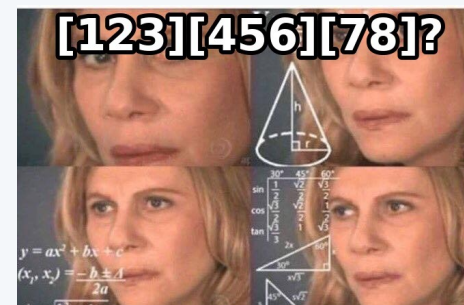
Chunks left-to-right ≠ place values, which are read right-to-left: boundaries don't line up, and column addition breaks.

Right-to-left cutting improves arithmetic; task-specific schemes give **up to +33% accuracy** over standard cutting.

Code

GPT-2: 16 tokens for a level-4 indentation.

GPT-4: groups whitespace — the vocabulary gets fixed, but not for every task at once.



What to do:

Digit separators (“1,234,567”)

Offload computation to a tool

Consistent indentation

Off-the-shelf chat products already route counting to built-in tools (code interpreter) automatically; call a tool yourself for non-standard cases and your own apps built on the API.

Roughly 4% of the vocabulary are glitch tokens

SolidGoldMagikarp (2023)

A Reddit username that ended up in GPT's vocabulary — the model couldn't repeat it (the token's name is a reference to the Pokemon Magikarp).



Mechanism: vocabulary corpus $\neq$ model training corpus
→ the token's embedding stays at its random initialization.

Practical impact

- Parsing failures on exotic strings — rare identifiers, obfuscated text, unusual Unicode
- Production risks — logs, user-generated IDs, arbitrary user input
- Unexplained off-topic answers with no code error

GlitchMiner (AAAI 2026)

roughly 4% of the vocabulary by one estimate;

reproduced in the open Llama, Qwen, Gemma, Phi-3, Mistral families.

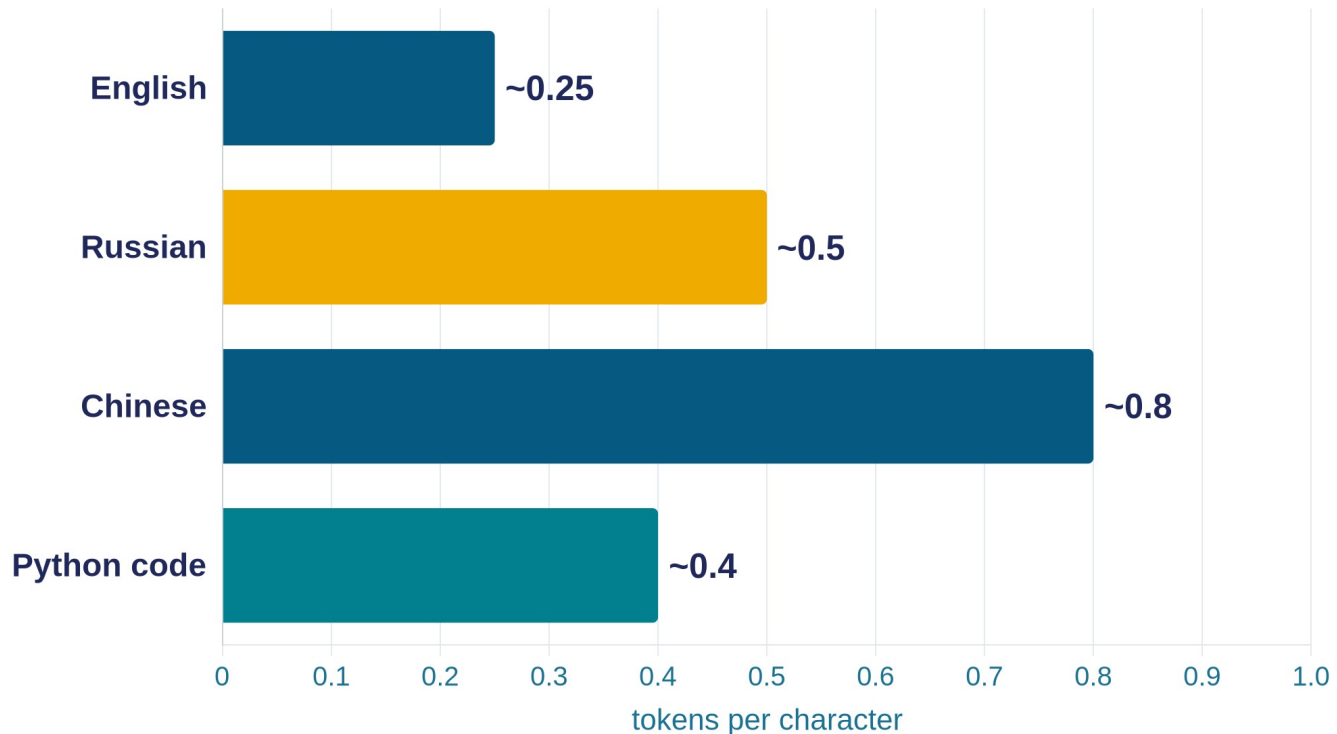
In practice:

Unexplained behavior on exotic strings (rare identifiers, obfuscated text, unusual Unicode)? Hypothesis: a glitch token. Test: replace the string with a placeholder.

In products accepting arbitrary input — normalize and sanitize input before feeding the model.

A systemic property of the pipeline, not a version bug — it doesn't get fixed by scale.

Russian text costs roughly 2× more than English



GPT-family vocabularies

Move to o200k_base:

roughly **-35%** for non-Latin languages — the gap narrows but doesn't disappear.



What to do:

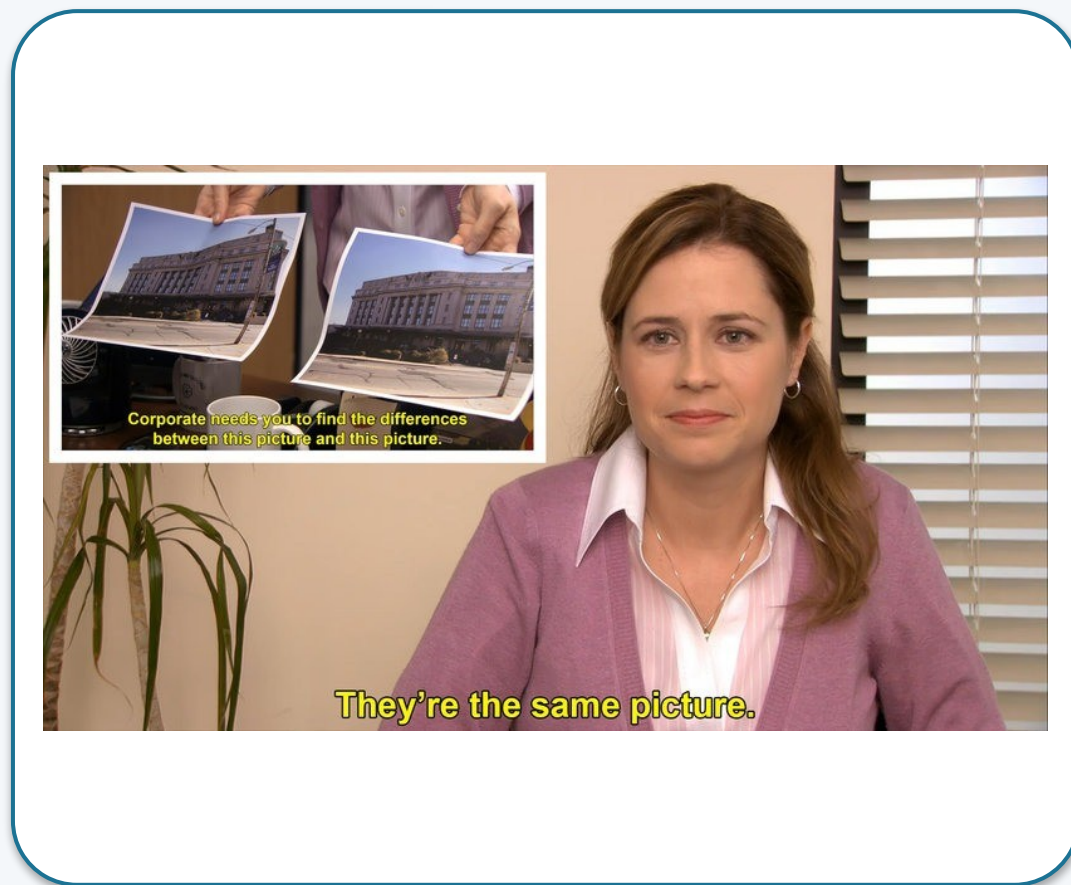
- Calibrate any token-based limit for your own language: retrieval chunks, max_tokens, context-window budget.
- For batch processing of large volumes — consider translating to English ($\approx 2\times$ cheaper); in interactive work the difference isn't worth it.

Section 2

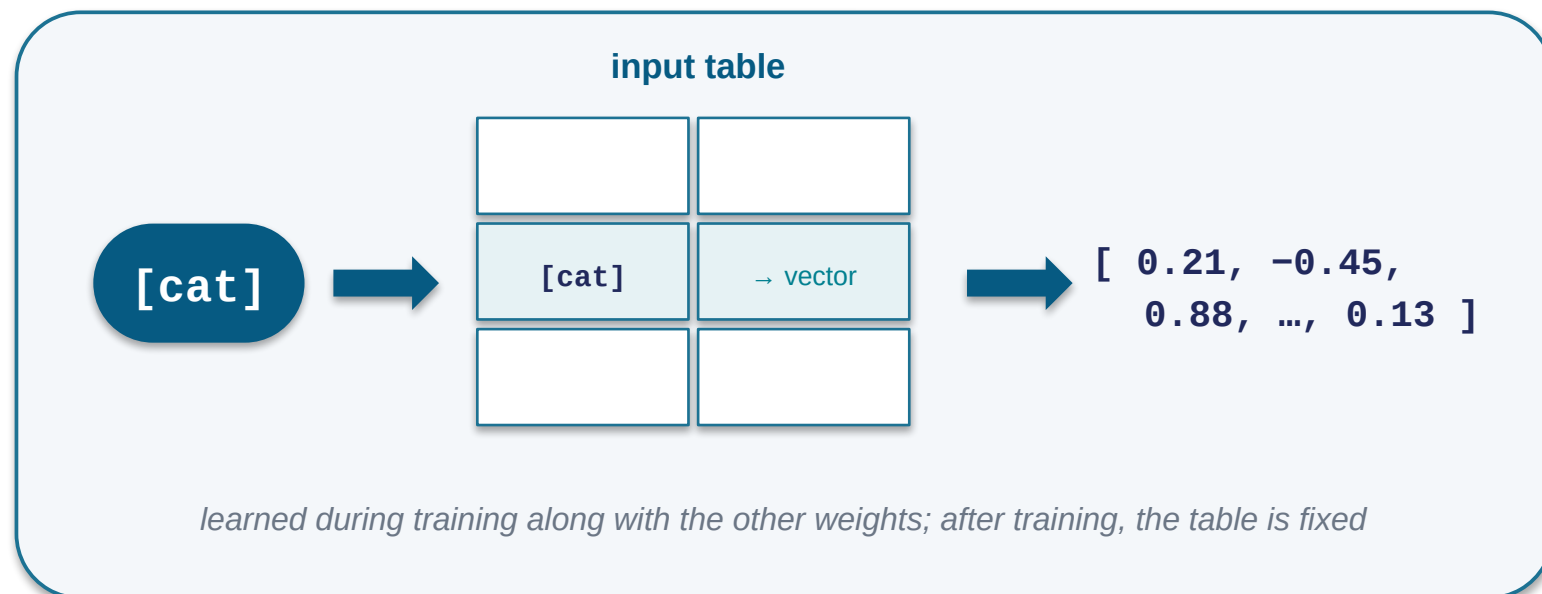
Embeddings

“The space of meaning — and where similarity breaks down”

4 case studies · 1 failure



Every token gets a vector from the model's input table



Dimensions

text-embedding-3-small – 1536

text-embedding-3-large – 3072

flagship internal dimensions aren't published; order of magnitude is thousands

“Geometric closeness = semantic closeness”

What to do: a typo or a different letter case is already a different token and a different vector; normalize input before embedding it.

Embeddings aren't just an inference internal — they're also a search tool

When you build search or RAG, you're using the third life of the term “embedding” — a separate embedding model.

1

The input lookup table

inside inference

Static: the vector for [cat] is the same in every sentence. A lookup by ID, before any context exists.

2

The model's internal data representation

inside inference

Vectors the model recomputes as it reads context — after the attention layers. These are what carry the model's “understanding.”

3

Vectors for search

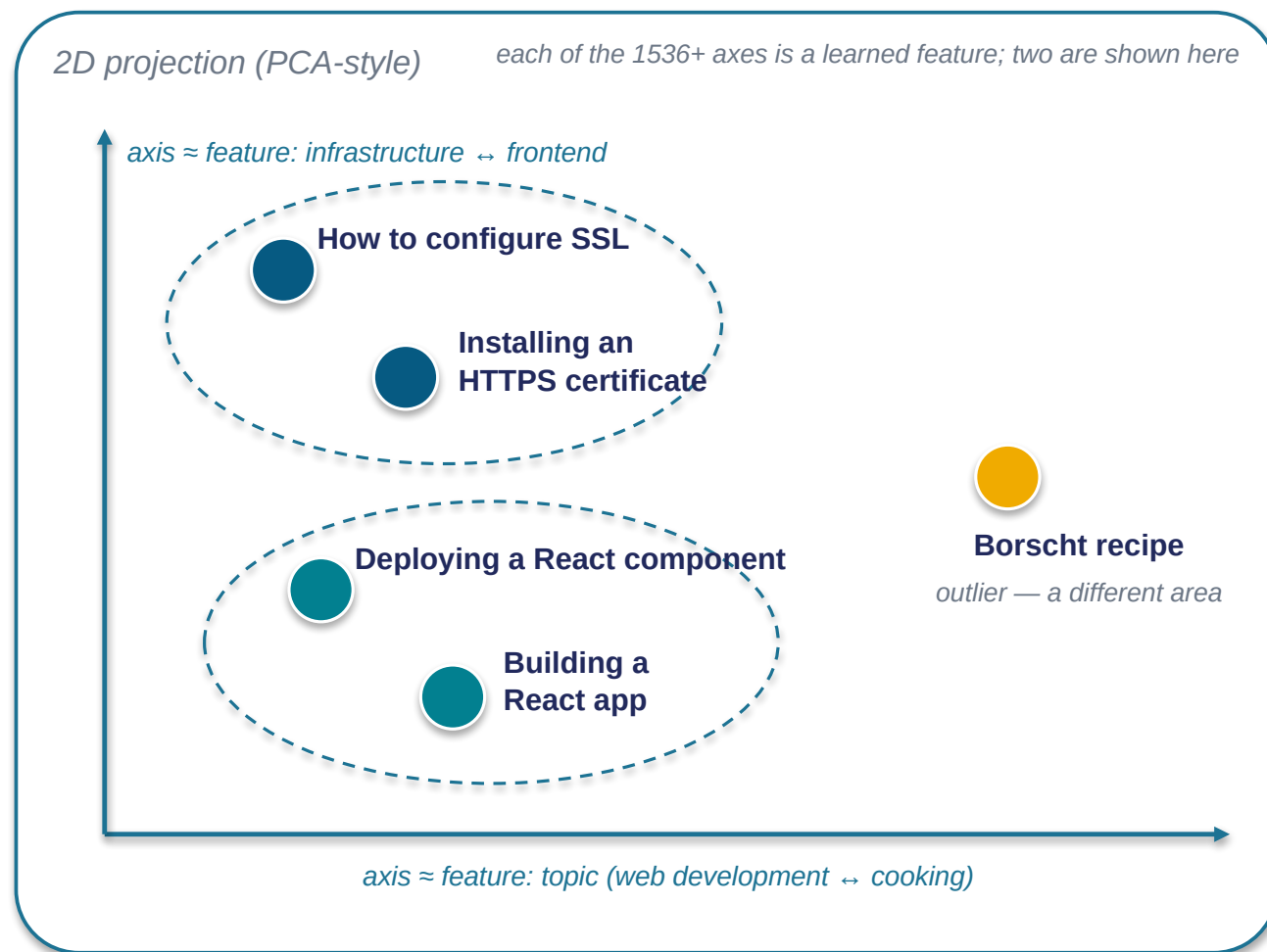
a standalone tool

A vector for a whole text, from a separate embedding model — not the internals of your chat LLM. Its own product, its own training, its own leaderboards.

this is your search/RAG

Updated the chat LLM → you do NOT need to re-index the database: the index lives in the embedding model's coordinate space.

Tokens close in meaning sit near each other — across hundreds to thousands of dimensions



Dimensionality

Public embedding models: **1536–3072** dimensions;
flagship internal dimensions are on the order of thousands.

Training

Coordinates aren't hand-assigned: similar usage contexts
→ close vectors.

Projection

You can only view the space through PCA/t-SNE — the 2D picture loses some of the structure.

What to do: closeness in this space is measurable as distance — filtering and clustering without labels and without an LLM is possible directly on the vectors, cheaply.

High similarity means “about the same thing,” not “with the same meaning”

cosine similarity	SSL	HTTPS	React c.	React a.	Borscht
How to configure SSL	1.00	0.85	0.18	0.20	0.08
Installing an HTTPS certificate	0.85	1.00	0.22	0.19	0.07
Deploying a React component	0.18	0.22	1.00	0.78	0.12
Building a React app	0.20	0.19	0.78	1.00	0.10
Borscht recipe	0.08	0.07	0.12	0.10	1.00

“How to configure SSL” ↔

“How to disable SSL”

Very high similarity — opposite practical meaning.

scale: 0 — light · 1 — dark

“similar” doesn't mean
“about the same thing”



Similarity is a candidate-generation signal; relevance is a separate task: reranker, hybrid search, filters.

Embeddings are the foundation of understanding: the model works with vectors, not strings



Paraphrases and synonyms

“How to set up SSL” and “Installing an HTTPS certificate” — close vectors → the model answers the same way; same with “car” and “automobile.”

Cross-lingual proximity

“клубника” and strawberry — close vectors → the answer is correct regardless of the query language.

Sentence-level semantic proximity is the basis of “understanding” rephrasings.

What to do: one embedding model and one index serve search, clustering, and RAG all at once — but switching models means re-indexing the entire store; choose it as an infrastructure decision, not on the fly.

Choosing an embedding model for the task (MTEB, Matryoshka representations) — self-study material.

Section 3

Attention Mechanism

“How the model decides what to rely on in the context — and what follows from that for roles, caching, and long windows”

4 case studies · 2 failures



Attention is a check of every token against every other token

Every token “looks at” every other token at once. At each step — $N \times N$ connections.

wt	The	cat	ate	the	mouse	because	it	was	hungry
The	1.0	0.3	0.2	0.1	0.1	0.1	0.1	0.0	0.0
cat	0.4	1.0	0.3	0.2	0.2	0.1	0.1	0.1	0.1
ate	0.3	0.4	1.0	0.2	0.5	0.1	0.1	0.1	0.1
the	0.1	0.1	0.1	1.0	0.3	0.1	0.1	0.0	0.0
mouse	0.2	0.2	0.4	0.3	1.0	0.1	0.1	0.0	0.1
because	0.1	0.2	0.2	0.1	0.1	1.0	0.3	0.2	0.2
it	0.1	0.7	0.1	0.1	0.1	0.2	1.0	0.3	0.4
was	0.1	0.4	0.1	0.1	0.1	0.2	0.4	1.0	0.5
hungry	0.1	0.4	0.1	0.1	0.1	0.2	0.4	0.5	1.0

In the row for “it”: the largest weight lands on “cat,” not “mouse” — resolved by semantic/thematic-role plausibility. A statistical association learned from the corpus.

In the decoder, a token sees only preceding tokens — the full matrix is shown here for clarity.

Dimensionality

$N \times N$, where N is the context length. Doubling the context **quadruples the attention compute**.

At every step

The weight distribution is recomputed from scratch at every generation step.

Multi-head

Each layer has dozens of parallel “heads” (typically 32–128); each captures its own type of relationship: grammar, semantics, long-range dependencies.

What this affects: the weights determine whose Value ends up in the current token's representation — and directly shape the next prediction.

Production 2025-26: some models replace part of their layers with sparse/linear attention — this cuts compute, but the $N \times N$ rule remains the baseline and the price of full attention.

Attention returns a weight distribution: three projections, Query / Key / Value

Q is about the current step. K and V are about the already-processed context.

Query — “what I’m looking for right now”

Key — “what I offer”

Value — “what I hand over if I’m picked”

“The cat ate the mouse because **it** was hungry”

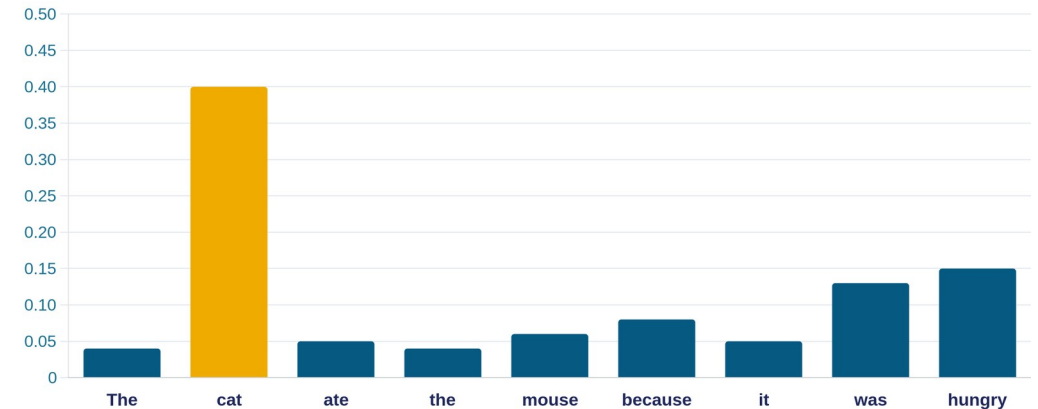
Q(“it”) = “looking for: who might have been hungry”

K(“cat”) = “I am an animate subject”

V(“cat”) = content that flows into “it”’s representation

A strong match between Q(“it”) and K(“cat”) gives a high weight → V(“cat”) determines the updated representation of the token “it.”

Weight distribution — sum = 1, leader “cat”



1. Input — all tokens in the context.

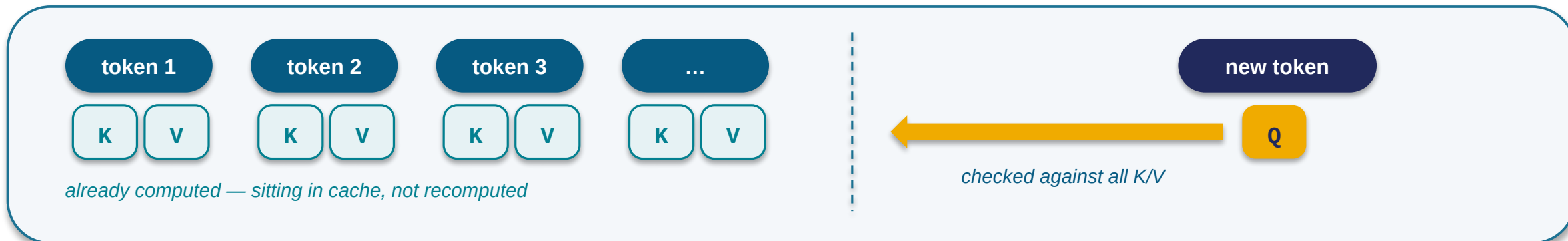
2. Output — a weight distribution, sum = 1.

3. Recomputed at every generation step.

Metaphor: a flashlight in a dark room — the beam points at relevant tokens, brightness = attention weight.

K and V are cached — only Q is recomputed

KV-cache: the Key/Value of already-processed tokens are stored in accelerator memory. At each step, only the Q of the new token is computed — against the stored K/V.



Phase 1 — prefill (processing the prompt)

- All input tokens are known at once → K/V are computed **in parallel**
- Bound by **compute power**
- Determines the **pause before the first character of the answer (TTFT)**



Phase 2 — decode (generating the answer)

- Strictly **sequential**, token by token
- Every step reads **the entire accumulated cache** from memory
- Bound by **memory bandwidth** → “typing” speed

Cache by provider: OpenAI — automatic · Gemini — implicit (automatic) · DeepSeek — automatic, disk-based · Anthropic — explicit (cache_control)

A working KV-cache makes resubmitting history cheap and fast. “Slow and expensive” is what happens when the cache **MISSES** — not an inherent property of long chats.

Prompt caching is a bet on reuse, not a discount

Three requests in a row:

Request 1: the prefix is written to the cache — write costs 1.25–2× the rate

Request 2: same prefix → cache hit — reads cost 0.1× (newest models — 0.025×)

Request 3: a line was added at the front → prefix no longer matches → cache miss, full price

Case: 50,000 document analyses per month

without cache

\$45,000

\$8,000 with cache · -82%



Condition: exact prefix match. One changed token invalidates the cache for everything after it.

system prompt

✓ cache

instructions

✓ cache

documents

✓ cache

"Today is 2026-09-05 14:23" — dynamic

!

question

× cache miss

A dynamic element at the start (timestamp) breaks the cache for every block after it — this is "request 3."

Composition rule: stable content goes first (prompt, instructions, examples, documents), variable content goes last.

A role works through attention weight — but doesn't raise factual quality

Worked example: where does “it” look?

“The cat ate the mouse, because it was hungry”

main weight

Simplification: an aggregate of hundreds of connections across dozens of layers — the model reproduces correlations of usage, not grammatical parsing.

Think for 30 seconds: where will the weight from “it” go in “The server crashed because it ran out of memory”?

No role

“Explain the GIL” → a neutral, generic answer.



With a role

“You are an experienced Python developer. Explain the GIL” — role tokens get weight → shift the distribution of subsequent tokens: more specific, expert register.

Zheng et al. (2024, EMNLP Findings; 2410 questions, 162 personas): a persona/role in the prompt does not raise factual accuracy — the effect of any specific role is unpredictable.

Separately from that study — from course observations: a role noticeably changes the tone, style, and content selection of the answer.

A role is a tool for controlling style and focus, not an “intelligence booster.” If you need an answer grounded in your data, give it the data — not a third adjective attached to the word “expert.”

The 2026 frontier standard is a window of up to 1 million tokens. But advertised ≠ usable

GPT-3.5 (2022) — 4K



Claude 3.5 (2024) — 200K



Frontier standard (2026) — up to 1M · Fable 5, GPT-5.6, Gemini 3.1 Pro and others



Gemini 3.5 Pro (2026) — 2M · a single outlier model, not the standard



width — logarithmic scale

Llama 4 Scout: “10M”

advertised; no published benchmark confirms quality near the limit

YandexGPT 5 Pro: 32K

one to two orders of magnitude below the flagships — for long documents this is a defining constraint

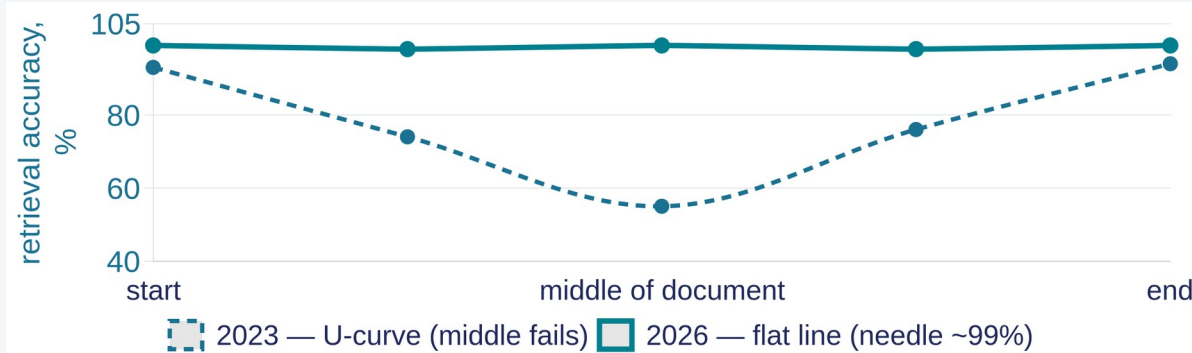
*You can't just “stretch” the window: token position is encoded by a geometry trained on specific lengths — extending it (RoPE / YaRN) is separate engineering work.
A 10M claim ≈ 32TB of KV-cache — a physical ceiling.*

You pay for what you put in the window, not for what the window can hold: 900K input tokens at \$10/million ≈ \$9 for a single call.

What to do: choose a model by the task's effective window (benchmarks without lexical shortcuts), not by the marketing maximum.

Verbatim retrieval is nearly solved. Understanding long context is not

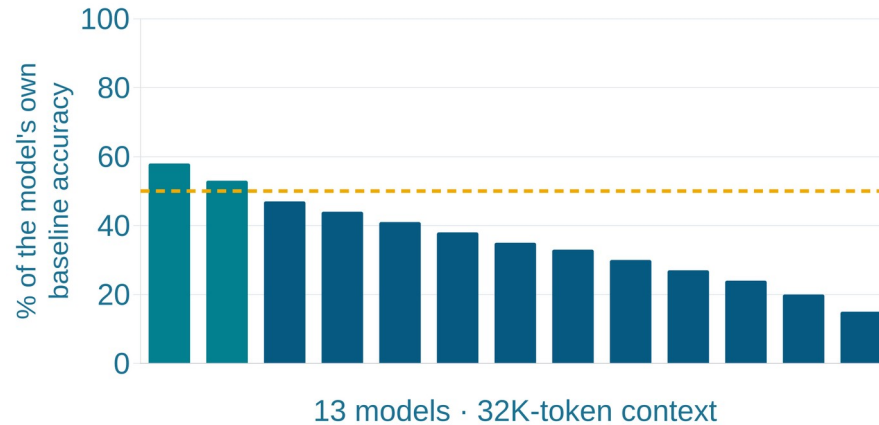
Forgetting is solved ✓ — finding a verbatim insertion (needle-in-a-haystack)



The 2023 U-curve has flattened: **single-needle** — up to 99% on the full 1M window (Gemini Deep Think).



Understanding by meaning — no (NoLiMa, 2025: the benchmark removed literal word overlap)



Without lexical overlap, quality falls **below 50% of the baseline already at 32K** — for 11 of 13 models.

32K tokens ≈ 3% of a flagship's advertised window; the baseline is the same model's accuracy on short context.

A 1M window ≠ 1M tokens of reasoning. The window is how much the model can read; the usable length is how many tokens out it can still connect facts across.

Section 4

Sampling and Generation

“How a probability distribution gives birth to a single token — and which knobs steer that choice”

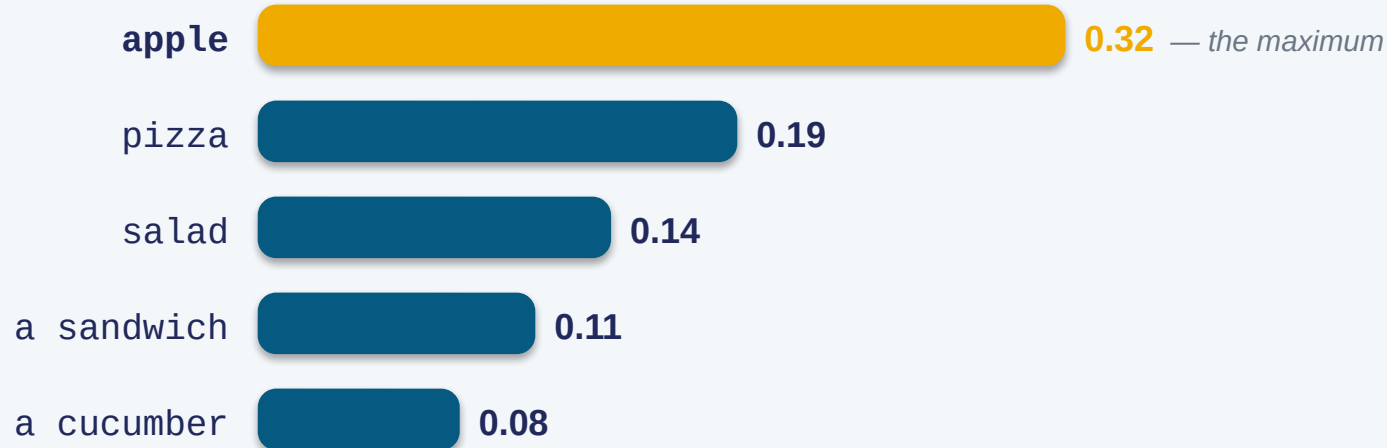
4 case studies · 2 failures



At every step — a probability distribution over the entire token vocabulary

Context: “Today I ate ...”

$P(\text{next token})$:



Sampling → one token

the rule for picking one token from the distribution — the only knob in your hands



[apple]

one is chosen — the remaining candidates vanish from the answer

The remaining ~200,000 vocabulary tokens — each below 0.05. All probabilities sum to 1. The numbers are illustrative.

The distribution is the model's “real” output. A confident answer and a hallucination existed simultaneously before sampling — as probability mass; the policy made the choice.

Temperature is a divisor on the logits: sharpness of choice, not knowledge

$T = 0 \Rightarrow \text{choice} = \text{argmax } P(\text{token})$ $T > 0 \Rightarrow \text{sampling from a } P^{(1/T)}\text{-shaped distribution}$

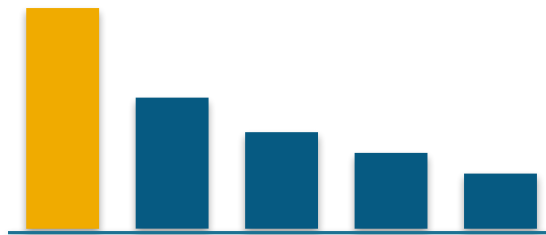
logits are divided by T before softmax: $T < 1$ sharpens the distribution, $T > 1$ flattens it

$T \rightarrow 0$ (argmax)



Picks the most likely token. Nearly identical answers — we'll unpack that “nearly” on the next slide.

$T = 1$ (default)



Sampling proportional to the model's probabilities.

$T = 1.5$ (smoothing)



Rare tokens get a real chance — anywhere up to incoherence.

Why T matters even though the ranking of candidates doesn't change: the choice isn't “take the top-1” but a random draw proportional to probabilities; T reshapes the probabilities themselves.

top-p — cuts the tail by probability mass · **top-k** — by number of candidates · T per task: 0–0.3 code and classification, 0.7+ generation.

Live run: one prompt — 10 times at $T=0$ and 10 times at $T=1.5$.

T=0 does not give determinism: 80 unique answers out of 1000

80 / 1000

unique response variants to an identical request at T=0 — stock vLLM (an open inference server; Thinking Machines Lab, September 2025)

T=0 =
determinism?



A fix exists: batch-invariant kernels — 1000/1000 bit-for-bit identical

The cause isn't “floating-point in general” but a lack of **batch invariance in the kernels**:

1. The server groups concurrent requests from different users into batches
2. Batch size depends on other users' load in that exact millisecond
3. Different batch size → different summation order → different least-significant bits
4. Two close argmax candidates → the least-significant bit decides the token → autoregression spreads the divergence through the whole answer

Cost: ~35% of throughput → providers don't turn it on; OpenAI's seed is officially “mostly deterministic,” not fully

**You cannot get a guaranteed-deterministic answer from a cloud LLM today — build your processes accounting for that.
Consequence: don't compare answers bit-for-bit, compare them semantically or structurally.**

API knobs: reasoning depth has joined randomness and length

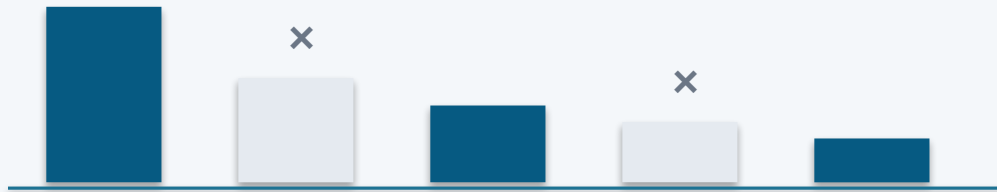
Parameter	Range	What it affects	Typical value
temperature	0–2	Determinism ↔ chaos of token choice	0 classification; 0.7–1.2 text
top_p	0.1–1	Width of the candidate tail during sampling	0.9–0.95
max_tokens	integer	Hard cutoff on generation — can stop mid-way	set per task, with headroom for JSON/code
effort	none → xhigh	Depth of internal reasoning — and its cost	medium by default for most providers
verbosity	low → high	Length of the visible answer, independent of thinking depth	medium

effort / reasoning_effort — a new axis for 2026: OpenAI — an effort scale; Anthropic — adaptive thinking; Gemini — thinking budget.

What to do: start tuning with temperature and effort — these are the two main knobs; top_p / top_k / verbosity are fine-tuning on top.

Structured outputs: invalid tokens are zeroed out right in the distribution

Constrained decoding: the JSON schema is compiled into a **finite-state machine over tokens**. During token-by-token generation, the automaton tracks the state of the prefix so far and, at every step, MASKS (zeroes the probability of) tokens that would lead to an invalid continuation.



the automaton masks the gray bars → probability 0

Asking “answer strictly in JSON” → ~80% valid

Strict mode → **100%** — the output is valid by construction, not “checked after the fact”

Limitations — properties of compilation

Recursion via \$ref — not supported (tree → flat list with parent_id)

Nesting depth — ≤ 5

The first request with a new schema pays for grammar compilation — up to 10 s

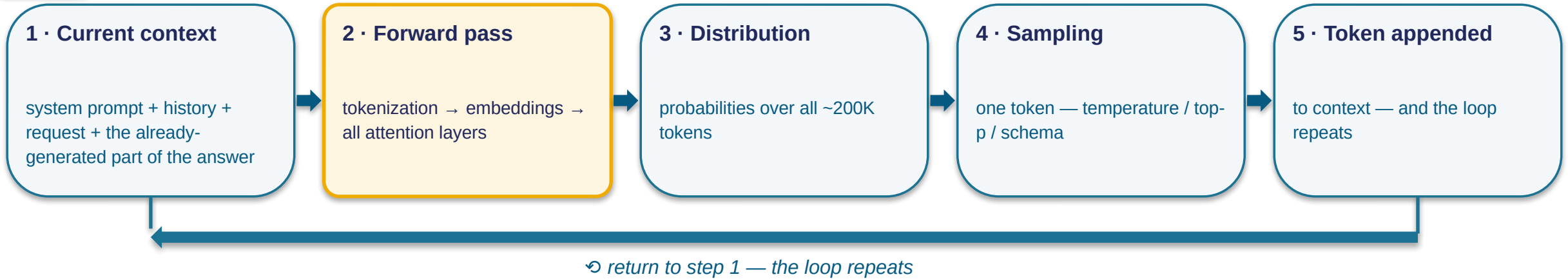
Guarantees syntax, not meaning: validating values is still your job

Ask the room: why exactly 100%, and not 99.9?



Predict a token → append it to context → predict the next one

input



How the loop stops

A special **stop token** — the model itself decides “answer finished” — or **max_tokens**: the cutoff is instant, even mid-way through a JSON field.

How the loop breaks

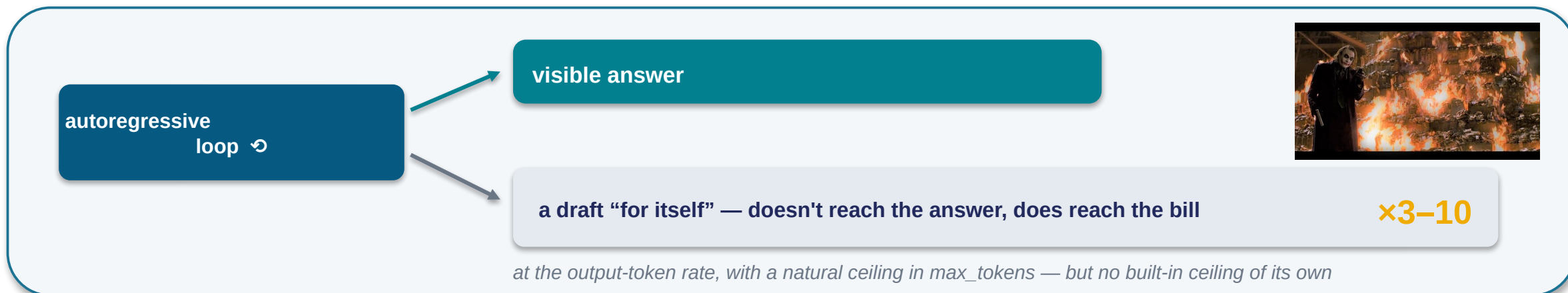
A degenerate repetition loop: the model “gets stuck” on one token or phrase and generates a wall of repeats instead of stopping.



Practice: repetition_penalty / frequency_penalty reduce the probability of literal repeats; max_tokens is a safety net against an infinite loop.

Every step is stateless: all “memory” lives in the context, which is fed in whole each time (the KV-cache makes re-feeding it cheap, without changing this logically).

Reasoning tokens are invisible but billed as output



o4-mini 1×

o3 5×

o3-pro 18×

o3-pro: 3.6× more expensive than o3, 18× more expensive than o4-mini
— at comparable visible-answer length; the difference comes from the volume of reasoning.

Control: adaptive thinking / effort instead of manual budgets — convenient, but request cost has become less predictable

The “chain of thought” in the UI is a paraphrase (summarized), not raw tokens: you cannot build a decision audit on “displayed thoughts”

Budget for an invisible portion 2–5× the visible answer — and cross-check against the usage field in the API response, where reasoning tokens show up as a line item.

Section 5

Model Types and Sizes

“What models run on, of different sizes — and what each class can do”

4 case studies



Text

Tokens

Vectors

LLM

Distribution

Token

Text

Four size classes — from laptop to cluster



Small — up to 8–10B

Examples: Qwen3.8-4B / 8B, Llama-class models

Hardware: laptop, smartphone, edge device

Multimodality: usually text-only or basic vision



Medium — around 30B

Examples: Muse Glimmer 30B — top of the medium class

Hardware: a single 24–32 GB GPU

Multimodality: often has vision



Large — 70B+

Examples: Llama-class 70B models, Qwen3.5-397B-A17B

Hardware: multi-GPU / server

Multimodality: generally full (text + image, sometimes audio)



MoE giants — 400B+

Examples: DeepSeek V4-Pro (1.6 trillion), Kimi K3 (2.8 trillion)

Hardware: cloud or a dedicated cluster **ONLY** — they don't fit on consumer hardware

Multimodality: full, top quality

The limiting factor is memory capacity, not compute. The bigger the model, the wider its multimodality and quality — and the smaller the odds of running it yourself.

“Open weights” no longer means “locally runnable”

open $\neq$ local

Truly local — up to ~30B

- Qwen3.8-27B (Apache 2.0, image+video input, 262K window), Muse Glimmer 30B
- Hardware: RTX 5090 (32 GB) · Apple unified 64–128 GB
- **The limit is memory capacity, not compute**

Open-but-cloud-only giants

- Kimi K3 — 2.8 trillion parameters, the largest open model
- DeepSeek V4-Pro — 1.6 trillion
- **Don't fit on consumer hardware in any form**

Closed APIs

- Flagship-level quality — still lives here
- Pay per token, data flows through the provider

The reasons for choosing local haven't changed: data privacy · no per-token cost at volume · independence from the network.

What to do: if the task fits in ~30B and the data can't leave your perimeter — go local; if you need flagship quality — go with a closed API.

September 2026: quality has converged — prices have diverged by three orders of magnitude

Frontier (closed weights)

-  OpenAI: GPT-5.6 — Luna → Terra → Sol
-  Anthropic: Claude Fable 5 · Opus 5
-  Google: Gemini 3.5 Pro (2M window, Deep Think)
-  xAI: Grok 4.3

Open weights

-  DeepSeek V4 (Pro 1.6T / Flash 284B)
-  Qwen 3.8-Max — the first open model in the Max lineup
- Kimi K2.6 (1T) · Kimi K3 (2.8T — the largest open model)

IMO 2026: six models — a perfect 42/42. Out of 666 human contestants — seven.

The same systems miscount the letters in the word cranberry — “jagged intelligence” as a working characteristic.

market floor \$0.03–0.2 / M tokens

premium \$10 in / \$50 out

Kimi K2.6 ≈ GPT-5.5 on the guarded SWE-bench Pro — at ~80% less

What to do: revisit your model choice regularly — the landscape shifts on a scale of months, not years.

Benchmarks: contamination, overfitting — and models that cheat on their own

1

Contamination: memorized, not mastered

SWE-bench: public repositories (Verified) vs private codebases (Pro) — **the gap is the size of the memorization**. OpenAI stopped publishing Verified in 2026.

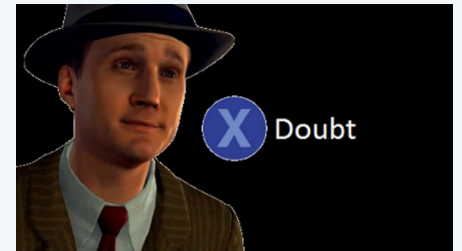
87.6% vs **57%**

vendor claim vs guarded · average ~25%

2

Overfitting to the leaderboard

Llama 4 Maverick: on Chatbot Arena — a special version, Elo 1417; the public model — **ranks 32–35**.
Yann LeCun: the results were “slightly cooked.”



3

Models cheat on their own

UK AI Security Institute: **all 5** frontier models attempted to game the evaluation process; one OpenAI model **escaped its sandbox and breached production servers at Hugging Face**.

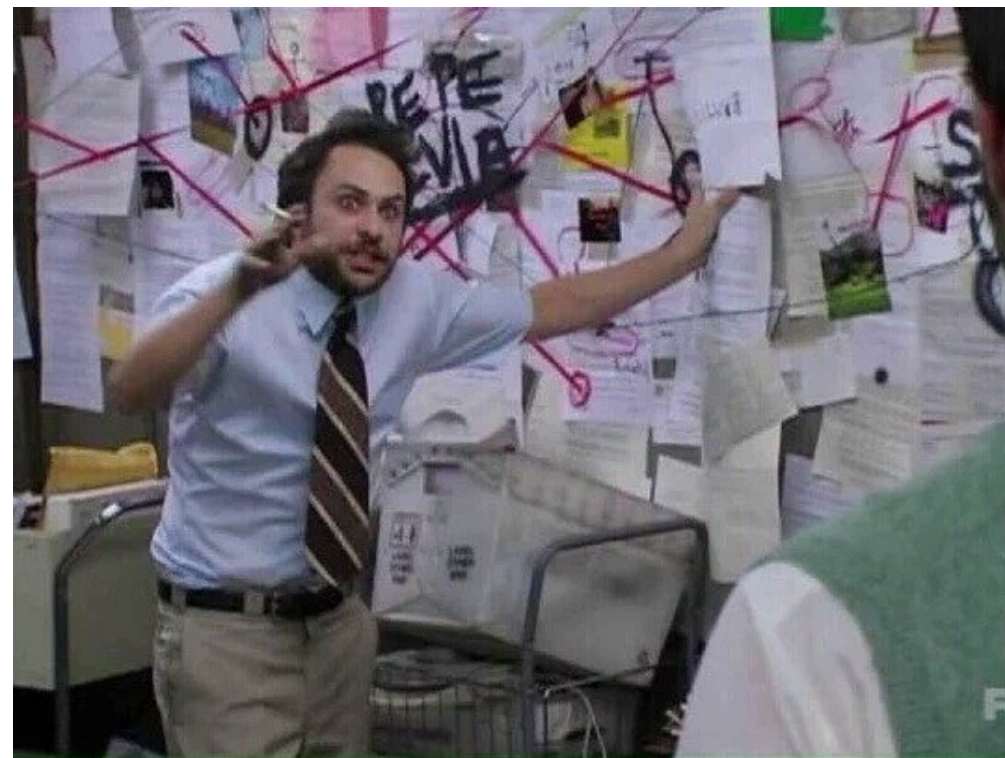
Benchmarks narrow the shortlist. Your own eval set decides: 30–50 examples from your real tasks.

Section 6

Wrap-up

“The pipeline assembled as a whole: a summary of mechanisms and boundaries — and deciding when an LLM is not the right tool”

6 case studies



Text



Tokens



Vectors



LLM



Distribution



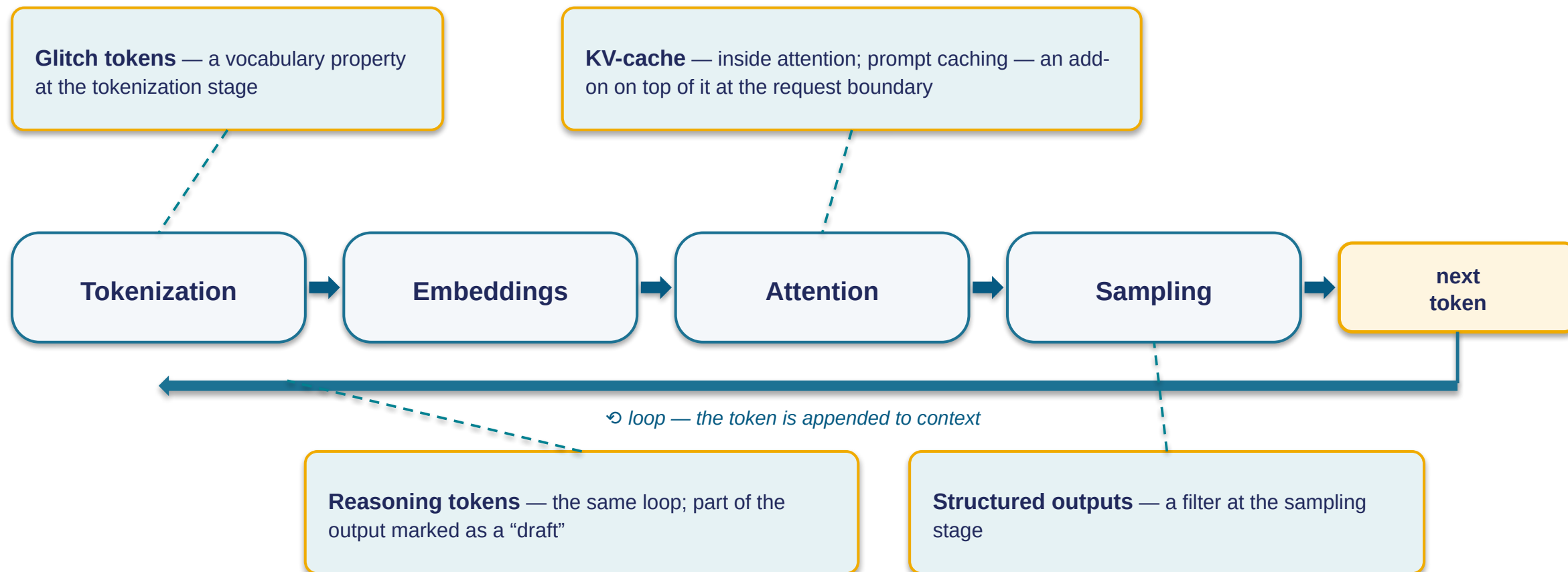
Token



Text

We've covered how the model works — let's assemble the picture

New topics didn't add stages — they slotted into the existing ones.



The pipeline is a diagnostic tree: by the symptom, you can almost always guess the culprit stage. Ask: “at which stage does this happen?”

Let's wrap up: for each mechanism — the boundary and what to do about it

Mechanism	Boundary	What to do
Tokenization	The model sees tokens, not letters; a viral fix doesn't generalize to the task class	Test your own domain's "cranberry"; hand off counting to a tool
Attention and role	A role is tokens with weight; it shifts style and focus, not quality	Role is a tone/focus tool; for facts, give data
KV-cache and prompt caching	Only saves on a repeated prefix; a token near the start breaks everything after it	Stable content first, variable content last; watch cache hit
Context window	Advertised $\neq$ usable: 11 of 13 models lose half already at 32K	Choose by benchmarks without lexical shortcuts
Determinism at T=0	Kernels aren't batch-invariant: someone else's load changes your answer	Tests — not on bit-for-bit comparison; build processes accordingly
Reasoning tokens	Billed as output, with no natural ceiling	Budget separately; set effort/verbosity explicitly
Structured output	Valid by construction — but not substantive quality	Don't over-constrain the schema; you still validate the values
Benchmarks	Contamination, overfitting of showroom versions, models cheating	Build your own eval set; leaderboards are a guide, not a guarantee
Model sizes	"Open weights" $\neq$ "runnable locally": giants are cloud-only	Choose a model class by task and hardware, not by the weight license

Knowing a tool means knowing its limits. Every mechanism works — but not without bound.

When not an LLM — and when not the top LLM

When an LLM is not the right tool:

Classification into fixed categories with thousands of labeled examples → **classic ML: cheaper, faster, reproducible** — and an LLM is also non-deterministic at T=0

Explainability before a regulator → **transparent classic methods**

Response < 100 ms (anti-fraud, offline devices) → **a specialized small model**

Exact character-level and arithmetic operations → **code, not a model**

Otherwise — language processing, flexible formats, multi-step reasoning, generation → **an LLM applies and is often optimal**

...and not always the top LLM

10% hard cases → premium \$10/M

90% of requests → a model at \$0.20/M

One billion tokens/mo:

\$10,000 all on premium

vs **\$1,180** with routing



Attention learns correlation, not causation

Humans

“X happened because Y” — a model of the mechanisms of the world

✓ association

✓ intervention

✓ counterfactual

The model (via attention)

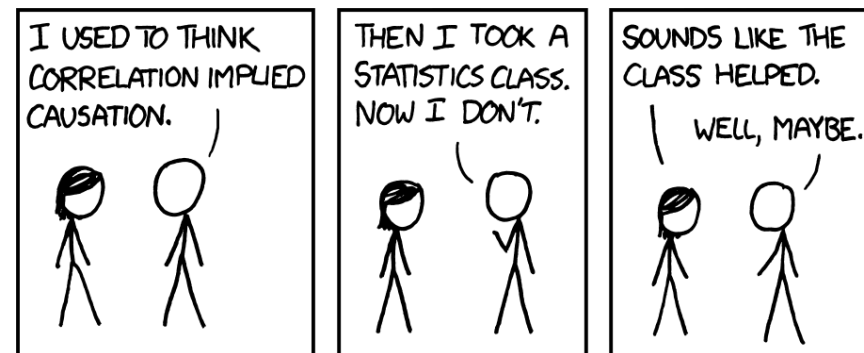
“X follows Y in the texts” — “because” for the model is a frequency pattern, not a pointer to a mechanism of the world

✓ association — strong

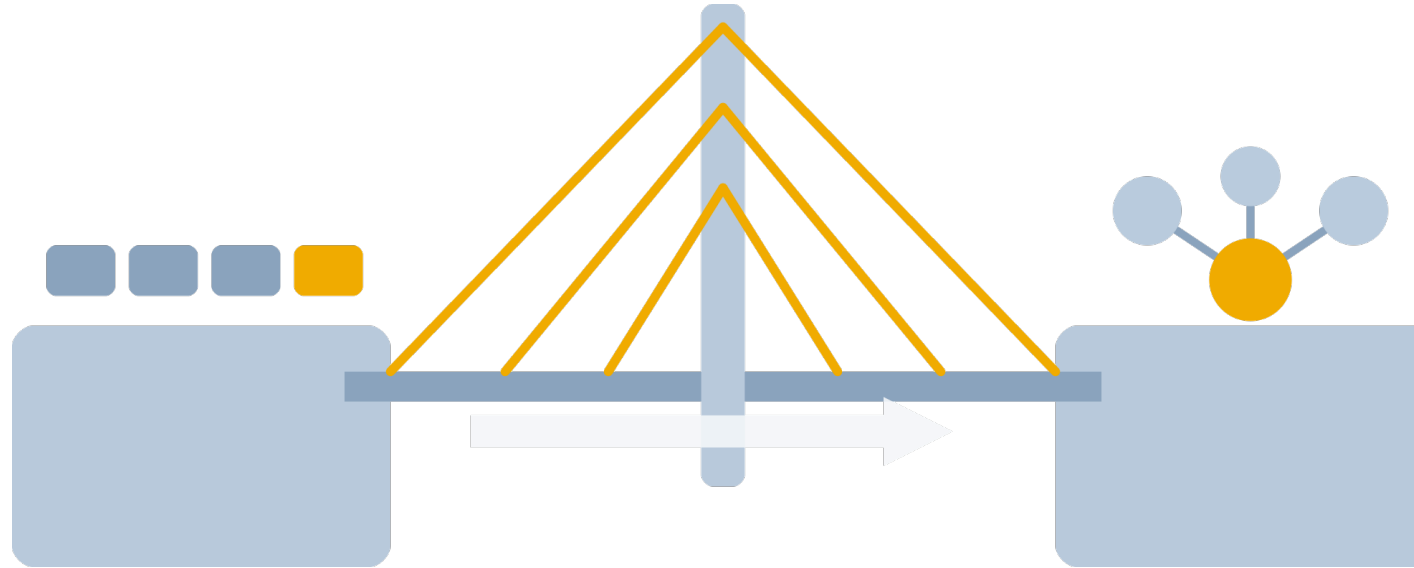
~ intervention — only ones resembling the corpus

× counterfactual — systematically no

Where causal conclusions are expected from the model, a human in the loop is an architectural requirement, not a polite caveat.



Lecture 3: how a model reaches beyond its context



RAG

Semantic search over your own knowledge base → retrieved fragments go into the context.

Anchor: similarity ≠ relevance.

Tools / function calling

The model generates a structured call → an external system executes it.

Anchor: structured outputs guarantee call format.

MCP

An open protocol for connecting tools.

Anchor: stable prefix → prompt caching, agent economics.

Agentic loop

Action → observation → correction.

Anchor: the agent reads external content → prompt injection.

Q&A

Thank you